



# 智能测试数据生成平台 —— 产品设计文档

## 一、现状与痛点

### 1.1 现状

当前，我们的测试工具链平台已支持七种类型的造数功能，具体包括：随机数造数、历史数据造数、历史规则造数、AI-表结构造数、AI-SQL脚本造数、AI-Mapping文档造数。其中，**AI-SQL脚本造数**功能的实现方式为：将测试所用的加工脚本 SQL 提供给 AI，由 AI 解析关联字段与脚本中使用的字段，进而开展关联造数与业务字段造数工作，以此辅助数据加工测试人员完成造数任务，提升测试质量与效率。

### 1.2 痛点

| 编号 | 痛点               |                                                                                |
|----|------------------|--------------------------------------------------------------------------------|
| 1  | AI-SQL 脚本造数存在天花板 | 当 SQL 脚本内容过大时，易超出 AI 调用的 token 限制                                              |
| 2  | 多种造数方式相互孤立，无法融合  | 平台上的多种造数方式各有其适用场景（AI-SQL 脚本造数适用于有复杂关联关系的场景；表结构造数适用于无关联字段的场景；历史规则造数适用于有固定规则的字段） |
| 3  | 规则配置工作量大，且易与预期不符 | 当前造数平台要求用户为每张数据表的每个字段填写造数规则                                                    |

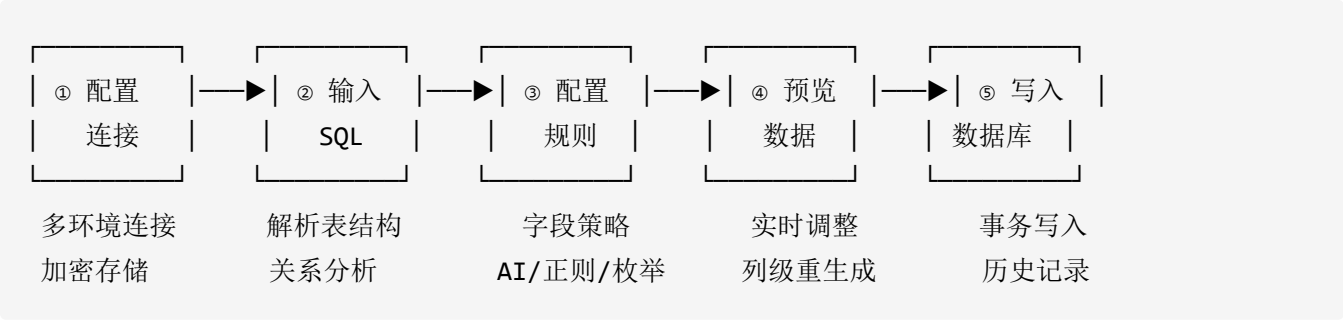
## 二、我们做了什么

**基于 AI 大模型的智能测试数据生成平台：**TestDataGen是一个基于AI的智能测试数据生成工具，核心目标是解决传统测试数据生成工具存在的痛点，通过结合人工智能大模型能力和数据库元数据解析技术，自动生成符合业务规则和数据库约束的高质量测试数据。

## 三、功能方案设计

### 2.1 整体工作流程

平台采用「五步向导」设计，用户从 SQL 粘贴到数据落库全程零编码：



### 2.2 各步骤功能详解

#### Step 1 — 数据库连接管理

- 管理多套目标数据库连接（开发/测试/预发各套环境）
- 一键连通性测试，配置项 AES 加密存储
- 连接信息（host、port、username、database\_name）统一管理，按需切换

#### Step 2 — SQL 输入与智能分析

输入支持：

- 手动粘贴业务 SQL（`INSERT INTO ... SELECT ...`、多表 JOIN `SELECT`）
- 文件上传（`.sql` / `.txt`），自动命名并保存到脚本库
- 脚本库：支持命名保存、重命名、删除，随时复用历史 SQL

自动分析内容（SqlParserService）：

| 分析项   | 说明                          |
|-------|-----------------------------|
| 涉及表清单 | 从 FROM、JOIN、子查询中提取所有表名      |
| 字段元信息 | 字段名、数据类型、最大长度、是否主键/自增/可空、注释 |

| 分析项      | 说明                                  |
|----------|-------------------------------------|
| 表间关系     | JOIN 关联、数据库元数据中的 FK 约束，精确到字段级       |
| 拓扑排序     | 依 FK 依赖关系排出生成顺序，保证子表写入时父表已存在        |
| WHERE 约束 | 自动提取 WHERE / JOIN ON 条件中的等值、IN、范围约束 |
| 循环依赖检测   | 出现循环 FK 时记录警告，原样追加以防死锁              |

WHERE 约束提取示例：

输入 SQL：

```
WHERE business_status IN ('ACTIVE', 'INACTIVE')
AND establish_date >= '2020-01-01'
AND establish_date <= '2024-12-31'
AND age > 18
```

自动推导出的规则建议：

- business\_status → 枚举规则 ['ACTIVE', 'INACTIVE']
- establish\_date → 范围规则 [2020-01-01, 2024-12-31]（同列的 >= 与 <= 自动合并取交集）
- age → 范围规则 min=18

可视化展示：

- 关联关系图：以 users.id → orders.user\_id (FK) 的连线方式展示
- 生成顺序：users → orders → order\_items（拓扑箭头）
- 表结构详情：字段名、类型、PK/AI/NN 标记、注释、关联引用关系

Step 3 — 字段规则配置

四种生成策略：

| 策略      | 标识              | 配置项    | 典型用途              |
|---------|-----------------|--------|-------------------|
| AI 语义生成 | LLM_DESCRIPTION | 语义描述文本 | 姓名、地址、公司名、订单备注等语义 |

| 策略    | 标识    | 配置项          | 典型用途              |
|-------|-------|--------------|-------------------|
| 正则表达式 | REGEX | 正则 pattern   | 身份证号、手机号、统一社会信用代码 |
| 范围值   | RANGE | min、max、type | 金额、年龄、日期区间、整数范围   |
| 枚举值   | ENUM  | 候选值列表（可加权重）  | 状态码、性别、省份、业务类型    |

**自动跳过字段：** 自增列（ `AUTO_INCREMENT` ）和关联引用列（由引擎自动处理，不需用户配置）。

**规则来源标记系统：** 每条规则都带有来源标识，帮助用户理解建议来自哪里：

| 标记             | 颜色 | 来源                  |
|----------------|----|---------------------|
| WHERE          | 黄色 | SQL WHERE/ON 条件自动推导 |
| HISTORY        | 蓝色 | 历史任务中相同字段的历史规则      |
| KNOWLEDGE_BASE | 紫色 | 知识库匹配建议             |
| AI             | 紫色 | 大模型分析建议             |
| COMMENT        | 灰色 | 字段注释自动识别            |
| MANUAL         | 灰色 | 用户手动修改              |

**规则配置辅助功能：**

- **自动回填：** 页面加载时自动调用 API，将历史规则 + 知识库建议批量回填，大幅减少手工配置
- **一键 AI 建议：** 针对整张表，调用大模型分析表结构（字段名、类型、注释、FK）后批量输出每列建议，直接回填到配置表单
- **字段历史规则：** 点击任意字段可查看历史使用记录（规则类型、配置、使用次数、最近使用时间），一键应用

## Step 4 — 数据预览与精细调整

**预览数据表格：**

| 行号 | 字段1 | 字段2(FK🔗) | 字段3(AI🔑) | 字段4(自增🔒) | ... |
|----|-----|----------|----------|----------|-----|
| 1  | 张伟  | 10001    | 已发货      | 1        | ... |
| 2  | 李娜  | 10003    | 待付款      | 2        | ... |

表头功能：

- 复选框选中列 → 批量重新生成
- 字段属性图标：🔒（自增，不可编辑）、🔗（关联字段，自动引用）
- 列菜单 (...)：重新生成此列 / 修改规则后重新生成 / 恢复原始数据

**单元格编辑：**双击进入内联编辑，Enter/失焦保存，Esc 取消，编辑后高亮标记"已修改"。

**对比与恢复：**开启 Diff 视图后，单元格下方显示"原: 旧值"；支持按列或整表一键恢复到初始预览状态。

**列级重新生成**（详见"功能亮点"章节）。

Step 5 — 写入数据库与任务记录

- 将预览数据（含所有手动编辑内容）批量写入目标数据库
- 全部表同一事务提交：任何一张表写入失败立即全部回滚，保证一致性
- 写入完成后自动创建任务历史记录，保存三类快照（规则 / 分析 / 数据）

任务详情页 Tab 结构：

| Tab    | 显示条件    | 内容                  |
|--------|---------|---------------------|
| 库表结构   | 始终      | 交互式 ER 关系图（可拖拽）     |
| SQL 语句 | 有输入 SQL | 只读 SQL 查看           |
| 造数规则   | 始终      | 按表分组的字段规则快照         |
| 写入数据   | 成功任务    | 每表最多 200 行数据快照，横向滚动 |
| 错误日志   | 失败任务    | 保留格式的错误信息           |

## 2.3 功能亮点与创新点

### 亮点一：简单高效的用户操作

传统测试数据工具往往需要用户逐字段配置生成规则，多表场景下配置工作量随字段数线性增长。本平台从三个层面系统性降低了用户操作成本：

#### 层面一：SQL 约束自动推导 — "SQL 即规则"

用户只需粘贴一条业务 SQL，平台即自动从 `WHERE` 和 `JOIN ON` 条件中提取语义约束，反向转化为字段生成规则，无需手工配置：

```
-- 用户输入的 SQL
WHERE business_status IN ('ACTIVE', 'INACTIVE')
  AND establish_date >= '2020-01-01'
  AND establish_date <= '2024-12-31'
  AND age > 18
```

平台自动推导：

- `business_status` → 枚举规则 `['ACTIVE', 'INACTIVE']` (IN 列表提取)
- `establish_date` → 范围规则 `[2020-01-01, 2024-12-31]` (同列 `>=` 与 `<=` 自动合并取交集)
- `age` → 范围规则 `min=18` (比较运算符提取)

支持的表达式类型：等值 `=` → 枚举、`IN (...)` → 枚举、`BETWEEN` → 范围、`>/>=`/`</<=` → 范围。同一字段多个范围条件自动合并（取最大下界 + 最小上界）。

#### 层面二：规则自动回填 + AI 一键推荐

进入规则配置页时，系统自动执行多层回填，用户大多数情况下只需确认即可：

回填优先级（由高到低）：

① 已保存规则

– 上次任务中用户手动确认过的规则

② WHERE 推导

– 从 SQL 约束自动推导（标记为"WHERE"来源，黄色标识）

③ 历史规则

– 其他任务中相同字段的规则（标记为"HISTORY"，蓝色标识）

④ 知识库匹配

– 企业知识库中的规则建议（标记为"KNOWLEDGE\_BASE"，紫色标识）

⑤ 启发式识别

– 内置规则库自动匹配（如 email 列 → 邮箱正则）

若自动回填仍不满意，用户可一键触发"AI 建议规则"，由大模型分析整张表结构后批量输出每列建议，直接回填到配置表单。每种来源都有独立颜色标记（WHERE/HISTORY/AI/COMMENT/MANUAL），用户对规则的出处一目了然。

层面三：预览即所得，一键写入

配置完成后点击"预览"即可看到生成效果，不满意可列级重新生成或单元格双击编辑，确认后一键写入数据库。全表事务原子提交，任意表失败自动回滚，用户无需关心写入细节。

亮点二：多元化造数规则支持

不同字段对数据的要求差异巨大：身份证号需要精确格式、订单备注需要语义丰富、金额需要数值范围、状态码只需枚举。单一策略无法覆盖所有场景，平台提供了"四种策略 + 五级匹配 + 列级重生成"的完整体系：

四种核心策略：

| 策略      | 精确度 | 典型用途           | 实现引擎          |
|---------|-----|----------------|---------------|
| AI 语义生成 | 语义级 | 姓名、地址、公司名、订单备注 | 大模型批量生成       |
| 正则表达式   | 格式级 | 身份证号、手机号、信用代码  | RgxGen 正则反向生成 |
| 范围值     | 数值级 | 金额、年龄、日期区间     | 区间均匀随机        |
| 枚举值     | 离散级 | 状态码、性别、业务类型    | 加权随机抽样        |

五级规则匹配优先级：系统为每个字段自动绑定最合适的生成器，匹配逻辑从高到低依次为：

- ① 关联字段列（最高优先） → `ForeignKeyGenerator`，自动从父表 PK 值池引用
- ② 用户规则 → 本次会议中用户明确指定的规则，精确匹配表名+列名
- ③ 存储规则 → 数据库中保存的规则，支持通配符模式（\* 匹配任意表/列）
- ④ 启发式规则 → 内置规则库自动识别：
  - 列名含 `email/mail` → 邮箱正则
  - 列名含 `phone/mobile` → 手机号正则
  - 列名 `== status` → 枚举 `["0","1"]`
  - 列名 `== gender/sex` → 枚举 `["M","F"]`
  - MySQL `enum()` 类型 → 自动提取枚举值
  - 列名含 `create_time/update_time` → 日期范围
- ⑤ 默认生成器（兜底） → 基于数据类型（`DataFaker`）生成基础值

对SQL脚本进行拆解析，提取脚本中的关联字段、WHERE条件等关键信息，通过代码实现关联字段的造数操作，确保关联字段造数成功率达到100%。

最后，基于“人机结合”理念，智能测试数据生成工具对重要功能进行拆分，确保重要字段造数成功率达100%；对于其他造数功能，则交由AI处理并设置兜底逻辑，保障测试数据生成成功。

**列级无损重新生成：**预览后发现某列数据不满意时，无需重新生成整张表。用户选中目标列，修改规则后仅该列重新调用生成器，其他列数据完全保持不变。关联字段列自动从已有父表数据中提取引用值，保证引用完整性。这种“局部刷新”能力让调整效率从  $O(\text{全表})$  降至  $O(\text{单列})$ 。

## 亮点三：高效可控利用大模型

**核心理念：**不是将库表结构和造数需求一股脑丢给大模型，而是通过 SQL 解析等预处理手段，将大模型调用收敛到“按字段维度”的精确调用，同时利用大模型的语义理解能力辅助规则推荐。

**问题背景：**一种直觉的 AI 造数方式是将“表结构 + 所有约束 + 生成行数”整体抛给大模型，让模型一次输出整张表的数据。这种方式存在明显问题：

- **成本高：**整表输出的 Token 消耗远大于单列输出， $10 \text{ 列} \times 100 \text{ 行} = 1000 \text{ 个值}$ 一次性生成
- **质量低：**模型需同时满足所有列的约束，出错概率随列数指数增长（如某列长度超限、类型不符）
- **不可控：**某一列出错需要整表重新生成，无法局部修正
- **不可组合：**正则、范围、枚举等确定性策略无法与 AI 输出混合编排

平台采用了"**预处理 + 按字段精确调用 + 规则引擎兜底**"的分层架构：

#### 第一层：SQL 解析预处理（SqlParserService）

- 解析表结构、字段类型、长度约束
- 提取 WHERE/ON 约束 → 自动推导为枚举/范围规则
- 识别表间关联关系 → 自动绑定 ForeignKeyGenerator
- 拓扑排序 → 确定多表生成顺序

产出：每个字段的"已知的确定性约束"已由规则引擎处理，只剩"需要语义理解"的字段才走大模型。

#### 第二层：规则引擎匹配（RuleMatchingEngine）

- 5 级优先级自动匹配：FK → 用户规则 → 存储规则 → 启发式 → 默认生成器
- 正则/范围/枚举/序列等确定性策略由对应 Generator 处理
- 仅 LLM\_DESCRIPTION 类型的字段才进入大模型调用流程

产出：一张表中通常只有部分字段需要调用大模型，其余字段由确定性生成器覆盖。

#### 第三层：按字段维度精确调用大模型（LlmService）

- 每次调用只针对单个字段，Prompt 精确包含：表名、列名、数据类型、长度约束、可空性、语义描述
- 单次请求返回 50 条值（可配置），存入线程安全值池
- 多个 LLM 字段并发调用，互不阻塞
- 多模型随机分配：不同字段可随机使用不同模型

优势：Prompt 聚焦单字段，约束明确、输出格式简单（JSON 数组），模型理解和遵守约束的成功率极高。

#### 第四层：容错降级（全链路兜底）

- API 调用失败 → 重试 3 次（指数退避）
- 全部重试失败 → DefaultGenerator 按数据类型兜底生成
- 并发异常 → 回退串行模式
- Markdown 响应 → 自动清理代码块标记

| 保证：任意场景下都能输出完整行数据，|

| 不会因 LLM 异常中断整个生成流程。|

具体对比：

| 维度            | 整表丢给 AI           | 平台的"预处理+字段级调用"    |
|---------------|-------------------|-------------------|
| 单次 Prompt 信息量 | 全表 10+ 列结构 + 所有约束 | 单字段定义 + 精确约束      |
| 输出格式          | 整表 N 行 × M 列 JSON | 单列 N 个值的 JSON 数组  |
| Token 消耗      | 高（输入+输出均大规模）      | 低（按字段拆分，每次仅单列）    |
| 约束遵守率         | 随列数增长而下降          | 单列约束简单，遵守率极高      |
| 出错影响面         | 整表需重新生成           | 仅影响单列，可列级重生成      |
| 确定性策略         | 无法与 AI 混合编排       | 正则/范围/枚举/AI 自由组合  |
| FK 处理         | 模型难以保证引用完整性       | 引擎自动追踪 PK，FK 精确引用 |
| 容错能力          | 模型失败则整表无数据        | LLM 失败回退默认生成器     |

AI 规则推荐 — 另一种高效利用方式：

除了数据生成，平台还利用大模型做"规则推荐"：将整张表的字段元信息（名称、类型、注释、PK/FK 标记）打包为结构化 Prompt，让模型分析每个字段最合适的生成策略，返回结构化 JSON 直接回填到配置表单。这种用法是"分析型"而非"生成型"——让 AI 做它擅长的事（理解语义、推荐策略），而不是让它做确定性程序更擅长的事（格式校验、范围约束、FK 引用）。

AI 规则推荐返回示例：

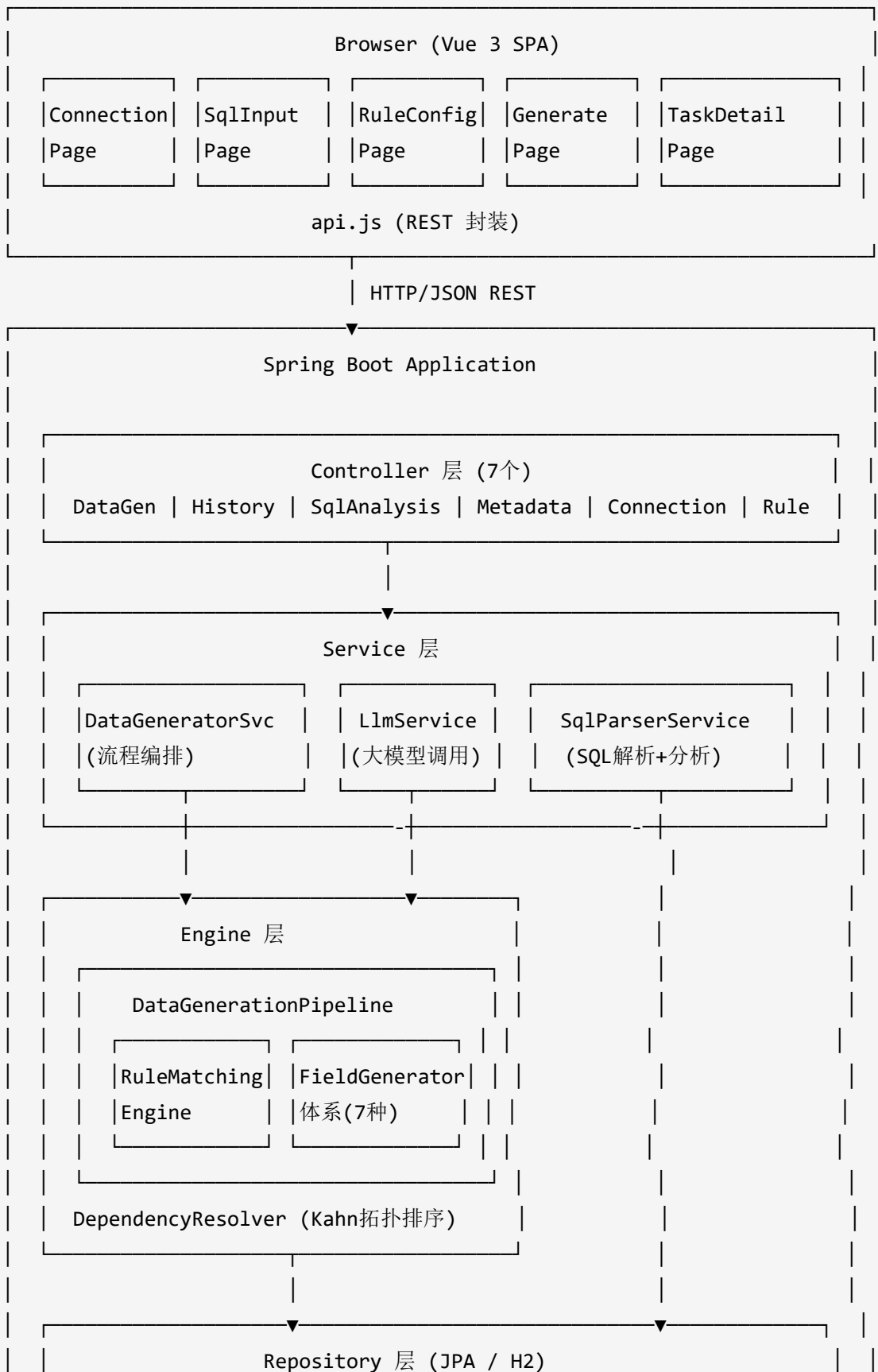
```
[
  {"columnName": "unified_code", "ruleType": "REGEX",
    "ruleConfig": {"pattern": "[0-9A-Z]{18}"}, "description": "18位统一社会信用代码"},
  {"columnName": "company_name", "ruleType": "LLM_DESCRIPTION",
    "ruleConfig": {"description": "中国境内真实企业名称，含有限公司、股份公司等后缀"},
    "description": "企业名称"},
  {"columnName": "business_status", "ruleType": "ENUM",
    "ruleConfig": {"values": ["正常", "注销", "吊销", "迁出"]}, "description": "企业经营状态"}
]
```

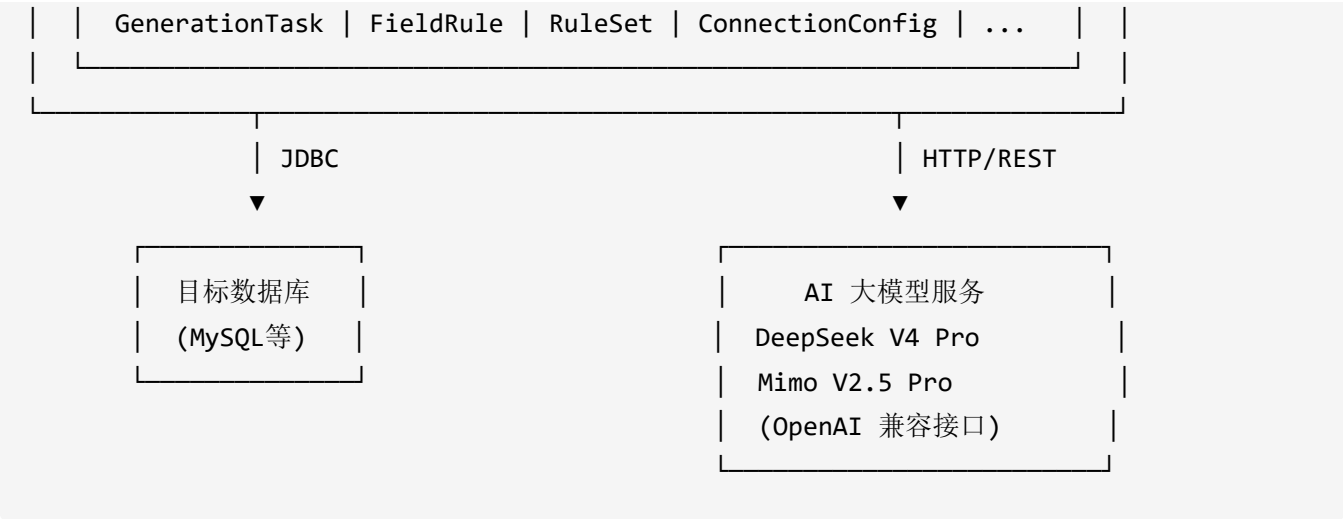
可以看到，AI 推荐的规则混合了 REGEX、LLM\_DESCRIPTION、ENUM 三种类型——这恰好体现了"让 AI 推荐策略，让引擎执行策略"的分工理念。

---

## 四、技术架构设计

### 3.1 系统整体架构





## 3.2 核心组件详解

### 3.2.1 数据生成引擎 (DataGenerationPipeline)

引擎是整个平台的核心枢纽，串联 SQL 分析结果、规则配置、LLM 服务和数据库写入。

三大核心方法：

① `preview()` — 纯预览（不写库）

`preview(DataGenRequest, SqlAnalysisResult) → DataPreviewResponse`

1. `buildDependencyLevels()`

└ 分析 FK 关系 → 返回分层列表 `[[table_A, table_B], [table_C], ...]`  
同层表之间无依赖关系，可安全并行

2. 按层级处理（层间串行，层内并行）：

FOR each level in levels:

IF `level.size() == 1`: `generateTablePreviewData()` 串行

ELSE: `CompletableFuture.allOf(...runAsync())` 并行，等所有完成

3. `generateTablePreviewData(tableName, ...)`:

└ `ruleMatchingEngine.matchRules()` → generators (5级优先级)

└ `setupForeignKeyGenerators()` → 注入父表已生成的 PK 值

└ `preFillLlmGenerators()` → 并发预填充所有 LLM 字段

└ `generateRows(rowCount)` → 逐行调用 `generator.generate(context)`

4. 维护 `generatedPkMap` (`ConcurrentHashMap`)

→ 追踪生成的主键值，供后续层级子表 FK 引用

5. 按 `generationOrder` 排序结果

→ 返回 `DataPreviewResponse {tableData, generationOrder}`

## ② `execute()` — 生成 + 事务写入

```
execute(DataGenRequest, SqlAnalysisResult) → ExecutionResult
```

1. 同 preview 流程并行生成所有表数据

2. 按 generationOrder 排列写入顺序（严格顺序）

3. 事务写入：

```
conn.setAutoCommit(false)
```

TRY:

```
FOR each table in allTableData (按 generationOrder 顺序):
```

```
    dataWriterService.writeRowsOnConnection(conn, tableName, rows)
```

```
    trackGeneratedPks() → 追踪自增列生成的实际 key
```

```
    conn.commit()
```

CATCH Exception:

```
    conn.rollback() ← 任何表失败，全部回滚
```

4. 收集数据快照（每表最多 200 行）

→ 返回 ExecutionResult {rowCounts, snapshotData}

### ③ regenerateColumns() — 列级无损重生成

```
regenerateColumns(tableName, columns[], rowCount, fieldRules, models,  
                  analysisResult, existingData) → RegenerateColumnsResponse
```

1. 过滤自增列（加入 warnings，跳过）

2. ruleMatchingEngine.matchRules() → generators

3. setupForeignKeyGeneratorsFromExistingData()

└ 从 existingData 中提取父表 PK 值注入 ForeignKeyGenerator

4. preFillLlmGeneratorsForColumns() → 仅对目标列中的 LLM 列预填

5. FOR i = 0 to rowCount-1:

```
    context = {rowIndex: i, currentRow: existingData[tableName][i]}
```

```
    FOR each targetColumn:
```

```
        value = generators[col].generate(context)
```

```
        value = truncateIfNeeded(value, colMeta)
```

```
        columnData[col][i] = value
```

6. 返回 RegenerateColumnsResponse {columnData, warnings}

### LLM 并发预填充机制：

```
preFillLlmGenerators(tableName, tableMeta, generators, totalRows, models)
```

收集所有 LlmBatchGenerator 类型的列

FOR each LLM列:

```
    CompletableFuture.runAsync(() -> {
        WHILE llmGen.needsMoreValues(needed):
            batchSize = min(llmBatchSize, needed)           // 默认批次 50 条
            description = 规则描述 || 字段注释 || 字段名
            values = llmService.generateBatchValues(...)
            IF values.isEmpty(): 用 DefaultGenerator 补充
            ELSE: llmGen.addValue(values)
    }, llmExecutor)    // 专用线程池: max(8, CPU×2)

CompletableFuture.allOf(...).join()    // 等待全部完成
CATCH: 回退到串行逐列预填
```

FieldGenerator 体系（7种实现）：

| Generator           | 依赖                 | 算法要点                                                       |
|---------------------|--------------------|------------------------------------------------------------|
| LlmBatchGenerator   | LlmService         | 预填值池（CopyOnWriteArrayList）+<br>循环取用（AtomicInteger），线程安全    |
| RegexGenerator      | RgxGen<br>2.0      | 将正则表达式反向生成匹配字符串                                            |
| RangeGenerator      | —                  | 按 type（int/long/double/date/datetime）在 [min,<br>max] 内均匀随机 |
| EnumGenerator       | —                  | 按权重数组进行加权随机抽样                                              |
| ForeignKeyGenerator | —                  | 从父表 PK 值池中 ThreadLocalRandom 随机引用                          |
| SequenceGenerator   | —                  | 从起始值按步长递增                                                  |
| DefaultGenerator    | DataFaker<br>1.9.0 | 基于数据类型兜底生成（varchar/int/date/decimal/<br>bool 等）            |

### 3.2.2 规则匹配引擎 (RuleMatchingEngine)

为每个字段绑定最合适的 `FieldGenerator`，采用 5 级优先级策略：

```
matchRules(tableName, columns, userRules, ruleSetId)
```

优先级 1（最高）：关联字段列

```
IF col.referencedTable != null → ForeignKeyGenerator（关联父键稍后由 Pipeline 注入）
```

优先级 2：用户规则（本次请求中传入的 `fieldRules`）

```
精确匹配: tableName.equalsIgnoreCase() && columnName.equalsIgnoreCase()
→ GeneratorFactory.create(ruleType, ruleConfig, dataType, maxLength, nullable)
```

优先级 3：存储规则（数据库中保存的规则，支持通配符模式）

```
glob 模式匹配: tablePattern && columnPattern && dataTypePattern
* → .* ; ? → . （转换为正则，CASE_INSENSITIVE）
```

优先级 4：启发式规则（内置规则库，按列名/数据类型识别）

- └ 列名含 "email"/"mail" → REGEX: [a-z]{5,8}@(gmail|qq|163|outlook)\.com
- └ 列名含 "phone"/"mobile" → REGEX: 1[3-9][0-9]{9}
- └ 列名 == "status" → ENUM: ["0","1"] 权重 [0.3, 0.7]
- └ 列名 == "gender"/"sex" → ENUM: ["M","F"] 权重 [0.5, 0.5]
- └ 数据类型 starts\_with enum(...) → 提取枚举值列表
- └ 列名含 "create\_time"/"update\_time" → RANGE: datetime [2024-01-01, 2025-12-31]

优先级 5（最低）：默认生成器

```
→ DefaultGenerator(dataType, maxLength, nullable)
```

### 3.2.3 SQL 解析服务 (SqlParserService)

基于 `JSQLParser 4.9` 对业务 SQL 进行深度语义解析：

解析层次：

输入 SQL

↓

CCJSqlParserUtil.parse(sql)

- ├ Insert 语句: 提取 target table + 内嵌 SELECT
- ├ Select 语句: PlainSelect / SetOperationList (UNION 等) / 子查询
- └ 解析失败: 正则回退提取表名

PlainSelect 解析流程:

- ├ FROM 子句: extractFromItem() → 表名 + 别名映射
- ├ JOIN 子句:
  - ├ getRightItem() → 右表 + 别名
  - ├ getOnExpressions() → 关联关系 + ON 条件提示
  - └ JOIN 类型: LEFT/RIGHT/INNER
- └ WHERE 子句: extractJoinRelation() + extractWhereHints()

## WHERE 提示提取支持的表达式类型:

AndExpression / OrExpression → 递归处理左右子树

EqualsTo {col = val} → ENUM: [val] (过滤纯列间关联条件)

InExpression {col IN (...)} → ENUM: 提取 ExpressionList (过滤子查询 IN)

Between {col BETWEEN a AND b} → RANGE: {min: a, max: b}

GreaterThan/GreaterThanEquals → RANGE: min

MinorThan/MinorThanEquals → RANGE: max

左右翻转处理: "5 > age" → effectiveOp 翻转为 "age < 5"

## 同列 RANGE 条件合并 (mergeRangeHints) :

- age >= 18 AND age >= 21 → 取最大下界 min = 21
- age <= 60 AND age <= 50 → 取最小上界 max = 50

## 拓扑排序 (DependencyResolver, Kahn 算法) :

构建有向图：

parent (被 FK 引用的表) → children (有 FK 的表)

inDegree[table] = 入度 (被依赖数量)

Kahn 排序：

queue = {inDegree == 0 的表}

WHILE queue not empty:

node = queue.pop() → result.add(node)

FOR each child of node:

inDegree[child]--

IF inDegree[child] == 0: queue.add(child)

循环依赖检测：

IF result.size() < tables.size():

剩余表追加到 result + warnings 中记录

---

### 3.2.4 可视化 ER 关系图 (SchemaGraph)

- 基于 HTML5 Canvas + 纯 JS 实现，无外部图形库依赖
- **贪心层次分配算法**：MAX\_LAYERS=5, MAX\_PER\_LAYER=4，避免所有表横向铺满
  - 长 FK 链 (A→B→C→...→H) 自动合并相邻层，保证不超出最大层数
  - 单层超过 4 个表时水平拆分为子列 (MAX\_SUB\_COLS=3)
- **节点拖拽**：onCardHeaderMouseDown 鼠标事件驱动，鼠标悬停提示"拖拽可移动表位置"
- 关联连线动态渲染，颜色按层区分

---

## 3.3 AI 大模型交互设计

### 3.3.1 两种 LLM 调用场景

场景 A：批量字段值生成 (核心高频调用)

触发时机：用户在规则配置中为字段选择了 LLM\_DESCRIPTION 策略，点击"预览"  
调用时机：DataGenerationPipeline.preFillLlmGenerators()，预览开始前并发调用  
每次调用：一次 API 请求返回 50 条值（llm-batch-size=50，可配置）

## Prompt 工程细节：

System Prompt（每次调用都携带）：

"你是一个严谨的数据库测试数据生成专家，必须严格遵守字段定义的所有约束（类型、长度、可空性、边界值等），绝不生成任何违反约束的值。"

User Prompt（动态构造，示例）：

"请根据以下字段定义生成50条realistic且diverse的测试数据，仅输出JSON数组，不包含任何其他说明。"

Table: t\_enterprise

Column: company\_name

Data type: varchar(200)

Max length: 200（若varchar类型则每个值字符数≤该值；若数值类型则忽略）

Nullable: false

Description: 中国境内真实企业名称，含有限公司、股份公司等后缀

额外要求：

- 若varchar(n)且n<36，严禁生成带连字符的UUID（应改用无连字符的32位十六进制、字母数字组合、雪花ID或符合长度的业务编码）
- 若bigint/bigint unsigned，注意非负范围
- 数据可以不唯一，但尽量多样化，且贴近description的语义，不要刻意使用边界值，尽量不要构造接近边界值的数据
- 输出格式：["北京华夏科技有限公司", "上海腾讯信息技术股份有限公司", ...]"

## 场景 B：表级规则推荐（低频辅助调用）

触发时机：用户在规则配置页点击"AI建议规则"按钮  
调用时机：RuleConfigPage → API.suggestRules()  
每次调用：一次 API 请求分析整张表的所有字段，返回每字段建议

## Prompt 工程细节：

User Prompt（动态构造）：

"Analyze this database table and suggest data generation rules for each column.

Table: t\_enterprise

Comment: 企业信息表

Columns:

- id (bigint) [AUTO\_INCREMENT] [PK]
- unified\_code (varchar(18)) -- 统一社会信用代码
- company\_name (varchar(200)) -- 企业名称
- business\_status (varchar(20)) -- 经营状态
- establish\_date (date) -- 成立日期
- user\_id (bigint) [FK->t\_user]

对于每个非自增字段，根据字段的数据类型和数据长度上限，给出最佳建议

Return JSON array: [{columnName, ruleType, ruleConfig, description}]

ruleType: REGEX (with pattern) | RANGE (with min/max/type) | ENUM (with values/weights)  
| LLM\_DESCRIPTION (with description)

请用中文回答，description字段请使用中文描述。"

### 3.3.2 多模型支持与随机分配

模型配置（application.yml）：

```

app:
  openai:
    # 全局默认配置
    model: deepseek-v4-pro
    max-tokens: 10000
    temperature: 0.8
    timeout-seconds: 60
    enable-thinking: true      # 开启 Extended Thinking

    # 各模型独立配置（可覆盖全局值）
    models:
      "[mimo-v2.5-pro]":
        base-url: https://token-plan-cn.xiaomimimo.com/
        api-key: ${MIMO_API_KEY}
        name: Mimo V2.5 Pro
      "[deepseek-v4-pro]":
        base-url: https://api.deepseek.com
        api-key: ${DEEPSEEK_API_KEY}
        name: DeepSeek V4 Pro

```

新增模型只需在 `models` 下添加条目，无需修改任何代码。

**模型随机分配**（`LlmService.pickModel`）：

```

// 用户可为同一任务选择多个模型
// 不同字段随机使用不同模型，提升生成结果多样性
private String pickModel(List<String> models) {
    if (models == null || models.isEmpty()) return openAiProperties.getModel();
    if (models.size() == 1) return models.get(0);
    return models.get(ThreadLocalRandom.current().nextInt(models.size()));
}

```

### 3.3.3 Extended Thinking（深度推理）支持

针对 DeepSeek 和 Mimo 两大模型系列的推理能力分别适配：

```
// DeepSeek 系列
if (actualModel.startsWith("deepseek") && enableThinking) {
    body.put("thinking", new JSONObject().put("type", "enabled"));
}

// Mimo 系列
if (actualModel.startsWith("mimo")) {
    body.put("thinking", enableThinking);
}
```

开启后模型先进行内部推理链再给出答案，对复杂约束场景（如字段间联动约束、多条件组合）生成质量更高。

### 3.3.4 容错与降级机制

| 异常场景         | 处理策略                                               |
|--------------|----------------------------------------------------|
| API 调用失败     | 自动重试，最多 3 次，指数退避（1s → 2s → 4s）                     |
| 全部重试失败       | 返回 <code>[]</code> ，LlmBatchGenerator 值池为空         |
| 值池为空时生成      | 回退到 <code>DefaultGenerator</code> 按数据类型兜底生成        |
| 并发预填异常       | 回退到串行逐列预填模式                                        |
| 响应含 Markdown | 自动去除 <code>```json</code> 和 <code>```</code> 代码块标记 |

任意场景下都能输出完整行数据，不会因 LLM 异常中断整个生成流程。

**3.3.5 LLM 调用完整链路图**

前端 RuleConfigPage / GeneratePage

|  
| ① 用户操作触发 (AI建议 / 预览按钮)

▼

DataGeneratorService.generatePreview()

|  
| SqlParserService.analyze() → SqlAnalysisResult

|  
| DataGenerationPipeline.preview()

|  
| preFillLlmGenerators()

|  
| CompletableFuture × N 个 LLM 字段 (并发)

▼

LlmService.generateBatchValues()

|  
| buildGenerationPrompt() ← 构造含约束的结构化 Prompt  
| pickModel(models) ← 随机选择模型

|

▼

POST /v1/chat/completions (OpenAI-Compatible API)

|                                        |
|----------------------------------------|
| System: 严谨测试数据生成专家                     |
| User: 字段定义 + 约束 + 格式要求                 |
| model: deepseek-v4-pro / mimo-v2.5-pro |
| thinking: enabled (推理增强)               |

|

▼

parseValuesFromResponse()

去除 Markdown → JSON.parseArray()

|

▼

LlmBatchGenerator.addValue(values)

→ valuePool (CopyOnWriteArrayList)

|

▼

generateRows() 时循环取用

generator.generate(context)

## 3.4 技术选型

| 层次      | 技术 / 库                     | 版本     | 选型理由                                                |
|---------|----------------------------|--------|-----------------------------------------------------|
| 后端框架    | Spring Boot                | 2.7.18 | 生产成熟，零配置快速启动                                        |
| 数据持久化   | Spring Data JPA + H2       | —      | 内嵌零配置数据库，DDL 自动维护 ( <code>ddl-auto: update</code> ) |
| SQL 解析  | JSQLParser                 | 4.9    | 工业级 Java SQL 语法解析器，支持复杂多层嵌套 SQL                     |
| AI 调用   | RestTemplate (OkHttp)      | —      | 兼容 OpenAI Chat Completions 接口，无厂商锁定，按模型独立配置         |
| 基础数据生成  | DataFaker                  | 1.9.0  | 多语言本地化假数据兜底 (姓名/地址/日期等)                             |
| 正则反向生成  | RgxGen                     | 2.0    | 将正则表达式反向生成符合模式的字符串                                  |
| JSON 处理 | FastJSON                   | 1.2.83 | 高性能 JSON 序列化，支持泛型 TypeReference                     |
| 前端框架    | Vue 3                      | —      | Options API，无构建步骤，SPA 嵌入静态资源直接部署                    |
| 并发      | CompletableFuture + 自定义线程池 | —      | LLM 并发调用，专用线程池 <code>max(8, CPU×2)</code>           |

## 3.5 接口设计

平台采用 RESTful API 风格，统一前缀 `/api/v1`，全部 JSON 通信。7 个 Controller 按业务域划分：

3.5.1 接口总览

| 模块     | 基路径                 | 职责                           |
|--------|---------------------|------------------------------|
| 连接管理   | /api/v1/connections | 数据库连接的 CRUD 与连通性测试           |
| 元数据    | /api/v1/metadata    | 目标库的表列表、字段元信息、关联关系           |
| SQL 分析 | /api/v1/sql         | SQL 语句解析，返回表结构/关联/拓扑/WHERE约束 |
| 脚本管理   | /api/v1/scripts     | SQL 脚本的保存、复用、删除              |
| 规则配置   | /api/v1/rules       | 字段规则 CRUD、AI 建议、历史规则、自动回填    |
| 数据生成   | /api/v1/generate    | 预览、写入、列级重生成、可用模型查询           |
| 任务历史   | /api/v1/history     | 任务列表、详情、数据快照、采纳率统计           |

3.5.2 核心接口详细设计

连接管理（ConnectionController）

| 方法     | 路径                | 说明     | 请求体               | 响应                   |
|--------|-------------------|--------|-------------------|----------------------|
| GET    | /connections      | 列出所有连接 | —                 | ConnectionConfig[]   |
| POST   | /connections      | 创建连接   | ConnectionRequest | ConnectionConfig     |
| PUT    | /connections/{id} | 更新连接   | ConnectionRequest | ConnectionConfig     |
| DELETE | /connections/{id} | 删除连接   | —                 | 204                  |
| POST   | /connections/test | 测试连通性  | ConnectionRequest | ConnectionTestResult |

```
// ConnectionRequest
{
  "name": "开发环境",
  "host": "192.168.1.100",
  "port": 3306,
  "username": "root",
  "password": "xxx",          // AES 加密存储
  "databaseName": "demo_db",
  "extraParams": "useSSL=false"
}
```

### SQL 分析 (SqlAnalysisController)

| 方法   | 路径           | 说明        | 请求体                 | 响应                |
|------|--------------|-----------|---------------------|-------------------|
| POST | /sql/analyze | 解析 SQL 语句 | {connectionId, sql} | SqlAnalysisResult |

```
// SqlAnalysisResult 响应结构
{
  "tableMetadataMap": {
    "t_user": {
      "tableName": "t_user",
      "columns": [
        {"columnName": "id", "dataType": "bigint", "maxLength": 20, "primaryKey": true, "autoIncrement": true},
        {"columnName": "name", "dataType": "varchar", "maxLength": 50, "nullable": false, "comment": "用户姓名"}
      ]
    }
  },
  "relations": [
    {"sourceTable": "t_order", "sourceColumn": "user_id", "targetTable": "t_user", "targetColumn": "id"}
  ],
  "generationOrder": ["t_user", "t_order"],
  "whereHints": [
    {"tableName": "t_order", "columnName": "status", "ruleType": "ENUM", "ruleConfig": "{\\\"values\\\": [\\\"待支付\\\", \\\"已支付\\\", \\\"已取消\\\"]}" }
  ],
  "warnings": []
}
```

规则配置 (RuleController)

| 方法     | 路径                                                 | 说明                    |
|--------|----------------------------------------------------|-----------------------|
| GET    | /rules                                             | 列出所有存储规则              |
| POST   | /rules                                             | 创建规则（支持通配符模式）         |
| PUT    | /rules/{id}                                        | 更新规则                  |
| DELETE | /rules/{id}                                        | 删除规则                  |
| POST   | /rules/suggest/{connId}/{table}                    | AI<br>建议规则（大模型分析整表字段） |
| GET    | /rules/history?tableName=&columnName=&sqlScriptId= | 字段历史规则                |
| POST   | /rules/auto-fill                                   | 批量自动回填（历史规则 + 知识库）    |

```
// auto-fill 请求体
{
  "sqlScriptId": 1,
  "tables": [
    {
      "tableName": "t_user",
      "columns": [
        {"columnName": "name", "dataType": "varchar(50)", "comment": "用户姓名"},
        {"columnName": "email", "dataType": "varchar(100)", "comment": "邮箱"}
      ]
    }
  ]
}

// auto-fill 响应: 按表返回规则列表
{
  "t_user": [
    {"columnName": "name", "ruleType": "LLM_DESCRIPTION", "ruleConfig": "{\\\"description\\\":\\\"中文姓名\\\"}"},
    {"columnName": "email", "ruleType": "REGEX", "ruleConfig": "{\\\"pattern\\\":\\\"[a-z]{5,8}@qq\\.\\.\\.c\\\"}" }
  ]
}
```

数据生成 (DataGenController)

| 方法   | 路径                           | 说明                      |
|------|------------------------------|-------------------------|
| GET  | /generate/models             | 获取可用 LLM 模型列表           |
| POST | /generate/preview            | 预览生成数据 (不写库)            |
| POST | /generate/execute            | 生成并直接写入数据库              |
| POST | /generate/write              | 将预览数据写入数据库 (含编辑/重生成元数据) |
| POST | /generate/regenerate-columns | 列级重新生成                  |

```

// preview / execute 请求体 (DataGenRequest)
{
  "connectionId": 1,
  "sql": "SELECT ... FROM t_user u JOIN t_order o ON ...",
  "rowCount": 100,
  "fieldRules": [
    {"tableName": "t_user", "columnName": "name", "ruleType": "LLM_DESCRIPTION", "ruleConfig": "{\
  },
  "models": ["deepseek-v4-pro", "mimo-v2.5-pro"],
  "sqlScriptId": 1
}

// preview 响应 (DataPreviewResponse)
{
  "tableData": {
    "t_user": [{"id": 1, "name": "张伟", "email": "zhangwei@qq.com"}, ...],
    "t_order": [{"id": 1, "user_id": 1, "amount": 299.00}, ...]
  },
  "generationOrder": ["t_user", "t_order"]
}

// write 请求体 (含编辑追踪元数据)
{
  "connectionId": 1,
  "sql": "...",
  "tableData": {...},
  "generationOrder": [...],
  "fieldRules": [...],
  "sqlScriptId": 1,
  "hasManualEdits": true,
  "hasRegeneration": true,
  "regeneratedColumns": {"t_user": ["email", "phone"]},
  "editedCellCount": 5,
  "regeneratedCellCount": 200,
  "totalCellCount": 1320
}

// regenerate-columns 请求体
{

```

```

    "connectionId": 1,
    "sql": "...",
    "tableName": "t_user",
    "columns": ["email", "phone"],
    "rowCount": 100,
    "fieldRules": [...],
    "models": ["deepseek-v4-pro"],
    "existingData": {...}
  }

  // regenerate-columns 响应
  {
    "columnData": {
      "email": ["new1@qq.com", "new2@163.com", ...],
      "phone": ["13800138001", "13900139002", ...]
    },
    "warnings": []
  }

```

任务历史 (HistoryController)

| 方法     | 路径                  | 说明           |
|--------|---------------------|--------------|
| GET    | /history            | 任务列表 (按时间降序) |
| GET    | /history/{id}       | 任务详情 (含三类快照) |
| GET    | /history/{id}/data  | 获取生成数据快照     |
| DELETE | /history/{id}       | 删除任务记录       |
| GET    | /history/statistics | 全局采纳率统计      |

```
// statistics 响应
{
  "totalTasks": 15,
  "totalCells": 19800,
  "editedCells": 45,
  "regeneratedCells": 600,
  "adoptionRate": 0.967,           // (total - edited - regen) / total
  "regenerationRate": 0.030       // regen / total
}
```

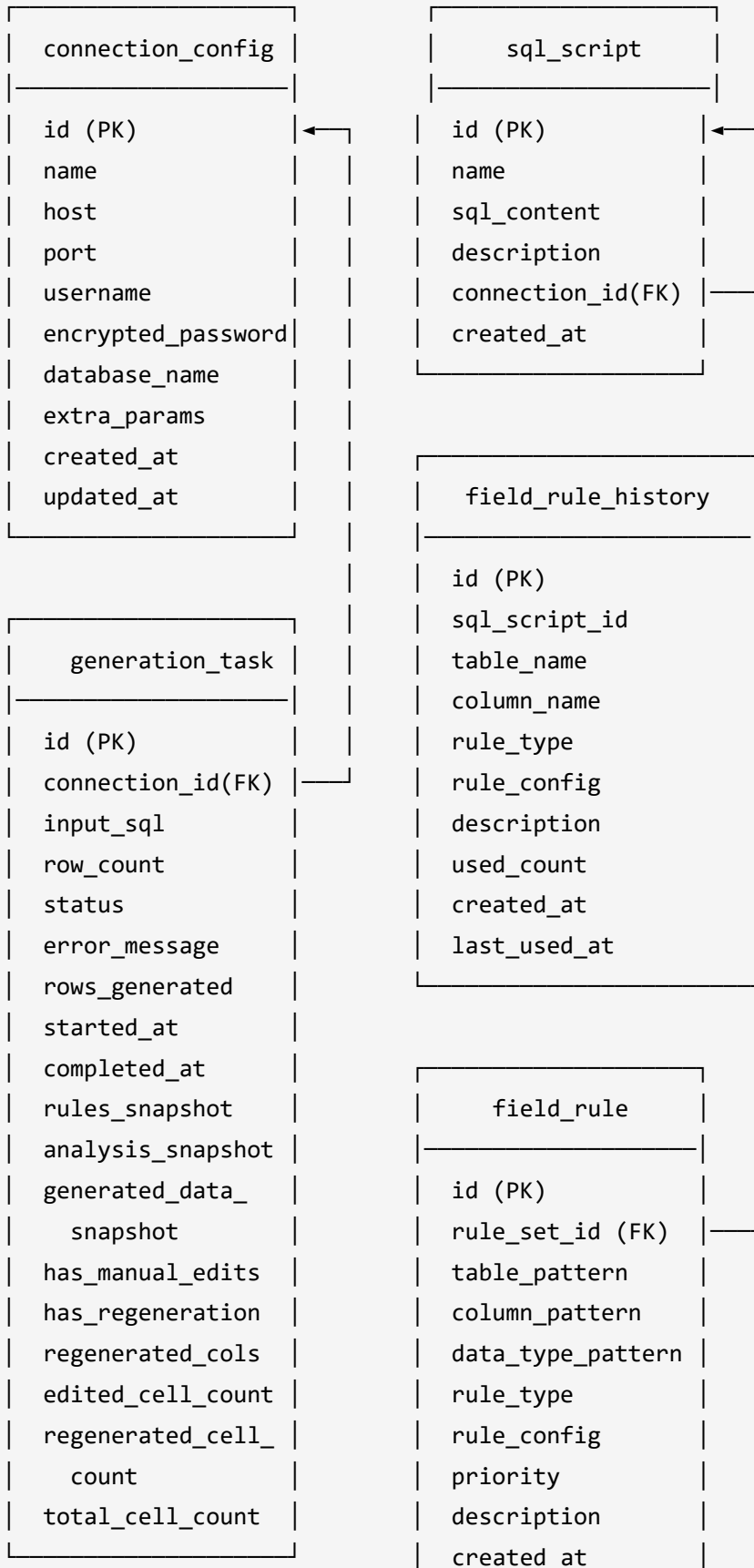
### 3.5.3 接口设计原则

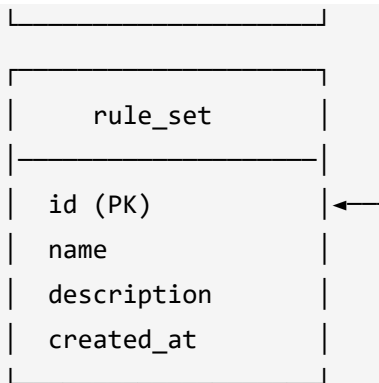
| 原则         | 实现方式                                                        |
|------------|-------------------------------------------------------------|
| RESTful 语义 | 资源名词化，HTTP 方法语义化（GET 查/POST 创/PUT 改/DELETE 删）               |
| 无状态        | 每次请求携带完整上下文（connectionId、sql、fieldRules），服务端不维持会话           |
| 统一错误格式     | Spring Boot 默认错误体 {timestamp, status, error, message, path} |
| 密码安全       | 连接密码 AES 加密后存储为 encryptedPassword，API 响应中不返回明文              |
| 快照保全       | 任务写入时保存规则/分析/数据三类快照，确保历史可追溯、可复现                             |
| 幂等安全       | GET/DELETE 幂等，POST 写入操作通过事务保证原子性                            |

## 3.6 数据库设计要点

平台使用 JPA `ddl-auto: update` 自动维护表结构，共 6 张核心表：

# ER 关系概览





## 核心表结构

### ① `generation_task` — 任务记录 + 三类快照 + 采纳率元数据

```
CREATE TABLE generation_task (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  connection_id BIGINT, -- 关联 connection_config.id  
  input_sql TEXT, -- 原始业务 SQL  
  row_count INT, -- 目标行数  
  status VARCHAR(20), -- PENDING/RUNNING/SUCCESS/FAILED  
  error_message TEXT, -- 失败原因  
  rows_generated INT DEFAULT 0, -- 实际生成行数  
  started_at DATETIME,  
  completed_at DATETIME,  
  rules_snapshot TEXT, -- 规则配置 JSON 快照  
  analysis_snapshot TEXT, -- SQL 分析结果 JSON 快照  
  generated_data_snapshot MEDIUMTEXT, -- 生成数据快照，每表 ≤200 行  
  has_manual_edits TINYINT(1) DEFAULT 0, -- 是否有手动编辑  
  has_regeneration TINYINT(1) DEFAULT 0, -- 是否有列级重生成  
  regenerated_columns TEXT, -- 重生成列信息 JSON {"table": ["col1", "col2"]}  
  edited_cell_count INT DEFAULT 0, -- 手动编辑的单元格数  
  regenerated_cell_count INT DEFAULT 0, -- 重新生成的单元格数  
  total_cell_count INT DEFAULT 0 -- 总可编辑单元格数（用于计算采纳率）  
);
```

### ② `connection_config` — 数据库连接配置

```
CREATE TABLE connection_config (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  name        VARCHAR(100) NOT NULL,      -- 连接名称
  host        VARCHAR(255) NOT NULL,      -- 主机地址
  port        INT NOT NULL DEFAULT 3306,  -- 端口
  username    VARCHAR(100) NOT NULL,      -- 用户名
  encrypted_password VARCHAR(512),        -- AES 加密后的密码
  database_name VARCHAR(100) NOT NULL,    -- 数据库名
  extra_params VARCHAR(512),              -- JDBC 额外参数
  created_at  DATETIME NOT NULL,
  updated_at  DATETIME NOT NULL
);
```

### ③ field\_rule — 存储规则（支持通配符模式匹配）

```
CREATE TABLE field_rule (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  rule_set_id BIGINT,                      -- 关联规则集
  table_pattern VARCHAR(200),              -- 表名模式 (* 匹配任意)
  column_pattern VARCHAR(200),            -- 列名模式
  data_type_pattern VARCHAR(100),          -- 数据类型模式
  rule_type    VARCHAR(30) NOT NULL,       -- REGEX/RANGE/ENUM/LLM_DESCRIPTION/SEQUENCE
  rule_config  TEXT,                      -- 规则配置 JSON
  priority     INT DEFAULT 0,              -- 优先级（同时匹配时取高）
  description  VARCHAR(500),
  created_at   DATETIME NOT NULL
);
```

### ④ field\_rule\_history — 字段历史规则（按使用频次追踪）

```

CREATE TABLE field_rule_history (
    id                BIGINT PRIMARY KEY AUTO_INCREMENT,
    sql_script_id     BIGINT,                    -- 关联脚本（可选）
    table_name        VARCHAR(200) NOT NULL,
    column_name       VARCHAR(200) NOT NULL,
    rule_type         VARCHAR(30) NOT NULL,
    rule_config       TEXT,
    description       VARCHAR(500),
    used_count        INT NOT NULL DEFAULT 1,    -- 使用次数
    created_at        DATETIME NOT NULL,
    last_used_at      DATETIME NOT NULL,
    INDEX idx_table_column (table_name, column_name),
    INDEX idx_script_table_column (sql_script_id, table_name, column_name)
);

```

## ⑤ sql\_script — SQL 脚本库

```

CREATE TABLE sql_script (
    id                BIGINT PRIMARY KEY AUTO_INCREMENT,
    name              VARCHAR(200) NOT NULL,
    sql_content       TEXT NOT NULL,
    description       VARCHAR(500),
    connection_id     BIGINT,                    -- 关联连接（可选）
    created_at        DATETIME NOT NULL,
    updated_at        DATETIME NOT NULL
);

```

## ⑥ rule\_set — 规则集分组

```

CREATE TABLE rule_set (
    id                BIGINT PRIMARY KEY AUTO_INCREMENT,
    name              VARCHAR(100) NOT NULL,
    description       VARCHAR(500),
    created_at        DATETIME NOT NULL
);

```

## 设计要点

| 设计决策       | 说明                                                                                                                                                                                                              |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 三类快照       | <code>rules_snapshot</code> / <code>analysis_snapshot</code> / <code>generated_data_snapshot</code><br>确保任务完全可追溯、可复现                                                                                            |
| MEDIUMTEXT | 数据快照可能较大（100行×20列），使用 MEDIUMTEXT（最大 16MB）                                                                                                                                                                       |
| 采纳率计算      | <code>adoptionRate = (total - edited - regen) / total</code> ，由 <code>*_cell_count</code> 字段支撑                                                                                                                  |
| 通配符规则      | <code>field_rule</code> 的 <code>table_pattern</code> / <code>column_pattern</code> 支持 <code>* / ?</code><br>通配，无需为每张表逐条配置                                                                                       |
| 历史规则去重     | <code>field_rule_history</code> 按<br>( <code>sqlScriptId</code> , <code>tableName</code> , <code>columnName</code> , <code>ruleType</code> )<br>唯一，重复使用时只更新 <code>used_count</code> 和 <code>last_used_at</code> |
| 密码加密       | 连接密码通过 <code>ENCRYPTION_KEY</code> 做 AES 对称加密，数据库只存密文                                                                                                                                                           |

## 3.7 并发与性能设计

| 优化点       | 实现方式                                                | 效果                            |
|-----------|-----------------------------------------------------|-------------------------------|
| 表间并行生成    | 同依赖层级表使用<br><code>CompletableFuture.allOf</code> 并行 | N 张无依赖表并行，时间从<br>O(N) 降至 O(1) |
| LLM 字段并发  | 每个 LLM<br>字段独立异步任务，专用线程池                            | M 个 LLM<br>字段并发请求，不互相阻塞       |
| 批量 LLM 调用 | 每次请求 50<br>条值（可配置），循环取用                             | 减少 API<br>调用次数，单次请求摊薄延迟       |
| 数据库批量写入   | <code>insert-batch-size: 500</code> ，批量<br>INSERT   | 单次 500<br>行，减少数据库往返           |

| 优化点      | 实现方式                                                                                                  | 效果         |
|----------|-------------------------------------------------------------------------------------------------------|------------|
| 线程安全数据结构 | <code>ConcurrentHashMap</code> ,<br><code>CopyOnWriteArrayList</code> ,<br><code>AtomicInteger</code> | 并发写入时无竞争冲突 |

## 五、质量验证

围绕"AI 能力"与"端到端功能正确性"两个维度对系统进行质量验证：

- **4.1 AI 能力测评：**量化两大核心 AI 能力的输出质量与模型表现
- **4.2 端到端业务场景验证（工商企业宽表）：**用一个高复杂度的真实业务场景，验证从 SQL 解析、依赖编排、规则生成到关联数据落库的全链路功能是否完整、严格生效

### 4.1 AI 能力测评

为了量化 AI 能力的真实表现并指导后续优化方向，对系统的两个核心 AI 能力（行级值生成、规则建议）建立了一套可复用的测评框架，并在小规模样本下完成了一轮基线评测。

#### 4.1.1 测评目标与方法

##### 测评目标

- 量化两大 AI 场景的输出质量，识别强项与短板
- 横向对比项目内置的两个模型，为默认模型选型提供数据依据
- 建立可复用的评测基线，便于后续迭代回归对比
- 暴露问题驱动后续规划（与"五、后期规划"形成闭环）

##### 测评对象（两大场景）

| 场景            | 入口方法                                          | 评测重点                     |
|---------------|-----------------------------------------------|--------------------------|
| 场景<br>A：字段值生成 | <code>LlmService.generateBatchValues()</code> | 单字段批量生成的合规率、语义贴合度、多样     |
| 场景            | <code>LlmService.suggestRules()</code>        | 字段语义识别、规则类型选择、ruleConfig |

| 场景     | 入口方法 | 评测重点 |
|--------|------|------|
| B：规则建议 |      | 正确性  |

评分维度与权重

| 维度    | 权重  | 说明                            |
|-------|-----|-------------------------------|
| 约束合规率 | 40% | 长度、类型、正则、枚举等硬约束通过率，最关键指标      |
| 语义相关度 | 25% | 生成值/推荐规则与字段语义的贴合程度（人工 1-5 分制） |
| 多样性   | 20% | 同字段内重复率、值分布均衡性                |
| 格式正确率 | 15% | JSON 解析成功率、字段完整性、无截断/乱码       |

综合评分公式

单场景得分

= 约束合规率 × 40 + 语义相关度 × 25 + 多样性 × 20 + 格式正确率 × 15

模型总分

= 场景A × 0.7 + 场景B × 0.3

等级划分

= S(≥90) / A(80-89) / B(70-79) / C(<70)

场景 A 权重更高的原因：值生成是最高频、最直接面向用户产物的能力。

4.1.2 测评框架（模板）

测试用例矩阵（场景 A：字段值生成）

| 用例编号 | 字段类型 | 字段示例                  | 约束特征  | 测试要点       |
|------|------|-----------------------|-------|------------|
| A-01 | 强格式  | phone (varchar 11)    | 正则约束  | 是否符合手机号格式  |
| A-02 | 枚举   | gender (char 1)       | M/F   | 是否落在枚举范围   |
| A-03 | 数值范围 | age (int)             | 0-150 | 是否落在合法区间   |
| A-04 | 中文语义 | name (varchar 20)     | 业务真实感 | 是否符合中文姓名特征 |
| A-05 | 长文本  | address (varchar 100) | 长度截断  | 超长是否被正确处理  |
| A-06 | 时间类  | birthday (date)       | 合理区间  | 与 age 是否一致 |

测试用例矩阵（场景 B：规则建议）

| 用例编号 | 字段名          | 字段类型         | 期望规则类型     | 测试要点     |
|------|--------------|--------------|------------|----------|
| B-01 | mobile       | varchar(11)  | REGEX      | 能否识别为手机号 |
| B-02 | id_card      | varchar(18)  | REGEX      | 能否识别为身份证 |
| B-03 | order_status | tinyint      | ENUM       | 能否给出业务枚举 |
| B-04 | email        | varchar(100) | REGEX      | 邮箱正则正确性  |
| B-05 | created_at   | datetime     | DATE_RANGE | 时间范围合理性  |

回退机制验证

- 强制断网/熔断状态 → 系统是否自动降级到内置规则生成
- 不合法 JSON 响应 → 是否触发解析失败兜底
- 长度超限 → 是否走截断兜底而非整批丢弃

4.1.3 测评结果

测评信息

| 项    | 值                                            |
|------|----------------------------------------------|
| 测评时间 | 2026-05 月度基线                                 |
| 样本规模 | 极小规模 — 场景 A 共 6×30=180 条样本，场景 B 共 5×3=15 次建议 |
| 参评模型 | deepseek-v4-pro（默认）、mimo-v2.5-pro            |
| 模型配置 | 来源 application.yml 实际配置                      |

场景 A 结果（字段值生成）

| 维度         | deepseek-v4-pro | mimo-v2.5-pro |
|------------|-----------------|---------------|
| 约束合规率      | 98.3%           | 88.9%         |
| 语义相关度（1-5） | 4.6             | 4.0           |

| 维度         | deepseek-v4-pro | mimo-v2.5-pro |
|------------|-----------------|---------------|
| 多样性（重复率反向） | 92%             | 78%           |
| 格式正确率      | 100%            | 96.7%         |
| 场景 A 得分    | 94.6            | 82.5          |

关键观察：

- deepseek-v4-pro 在 A-04 中文姓名、A-05 长文本上表现稳定
- mimo-v2.5-pro 在 A-02 枚举上偶现"男/女"中英混用，需后置归一
- 两个模型在 A-06 时间类上语义合理但与 age 一致性不足（→ 5.1 行级关联）

场景 B 结果（规则建议）

| 维度                   | deepseek-v4-pro | mimo-v2.5-pro |
|----------------------|-----------------|---------------|
| 约束合规率（ruleConfig 可用） | 93.3%           | 80.0%         |
| 语义相关度（1-5）           | 4.4             | 3.8           |
| 多样性                  | 86%             | 72%           |
| 格式正确率                | 100%            | 93.3%         |
| 场景 B 得分              | 85.9            | 75.4          |

关键观察：

- B-02 身份证：deepseek 给出含校验位的完整正则，mimo 仅给 18 位长度
- B-03 订单状态枚举：两个模型均无法准确给出业务枚举（→ 5.3 知识库）

模型对比与综合评分

| 模型              | 场景 A | 场景 B | 总分   | 等级 |
|-----------------|------|------|------|----|
| deepseek-v4-pro | 94.6 | 85.9 | 92.0 | S  |
| mimo-v2.5-pro   | 82.5 | 75.4 | 80.3 | A  |

回退机制验证结果

| 验证项           | 结果   |
|---------------|------|
| 网络异常自动降级到规则生成 | ✓ 通过 |
| JSON 解析失败兜底   | ✓ 通过 |
| 超长数据截断兜底      | ✓ 通过 |
| 切换备用模型        | ✓ 通过 |

#### 4.1.4 测评结论与改进方向

##### 结论

- **默认模型选择合理**：deepseek-v4-pro（S 级 92.0）综合表现优于 mimo-v2.5-pro（A 级 80.3），符合配置默认值
- **mimo-v2.5-pro 作为备份模型可用**：A 级表现满足故障切换需求
- **整体能力达标**：约束合规与回退机制满足生产可用门槛

##### 暴露的问题及对应改进方向

| 测评发现的问题                        | 对应改进方向                           |
|--------------------------------|----------------------------------|
| 同行字段语义错位（如 birthday 与 age 不一致） | → <b>5.1 多字段语义关联生成</b>           |
| 业务枚举类字段（订单状态、支付状态）建议质量差        | → <b>5.3 知识库增强</b> （业务枚举沉淀）      |
| 部分字段（A-02 枚举、B-02 身份证）模型间表现差距大 | → <b>5.2 历史采纳数据反哺</b> （按字段动态选模型） |
| 知识库覆盖面窄，常见字段每次都靠 LLM 现场推       | → <b>5.3 知识库增强</b> （标准字段词典）      |

测评结果直接为下一阶段的规划方向提供了数据支撑。

### 4.2 端到端业务场景验证（工商企业宽表）

AI 测评聚焦"单点能力"，而真实业务场景检验的是**完整工具链**：SQL 解析 → 多表依赖排序 → 规则生成 → 关联字段对齐 → 批量数据落库的全流程。为此专门构建了一个高复杂度的银行业"全量工商企业信息宽表"场景（`src/main/resources/sql/enterprise-data-scenario.sql`）作为端到端验证基线。

4.2.1 场景背景

- **业务来源**：模拟银行/金融机构对外部工商数据进行**多源融合**，将 7 张来源系统表加工为一张全量工商企业信息宽表（`t_full_enterprise_info`），用于下游风控、客户画像、营销推荐等场景
- **验证目标**：覆盖系统所有核心功能点的"压力测试用例"，确保设计的关联关系、约束、生成顺序等在复杂场景下**严格满足**而非"勉强能跑"

4.2.2 表复杂度

| 维度      | 数据                                                                                                                                              |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 来源表数量   | 7 张（ <code>t_upstream_code</code> 、 <code>t_external_code</code> 、 <code>t_gs_key</code> 、 <code>t_legal_person_cert</code> 、 <code>t_cu</code> |
| 目标宽表    | 1 张（ <code>t_full_enterprise_info</code> ），共 <b>80+ 字段</b>                                                                                      |
| 核心源表字段数 | <code>t_enterprise_base</code> 50+ 字段； <code>t_merchant_base</code> 28 字段；其余源表均 13-20 字段                                                        |
| 数据类型多样性 | 覆盖 8 种类型： <code>VARCHAR(8~1024)</code> 、 <code>CHAR(1)</code> 、 <code>INT</code> 、 <code>DECIMAL(18,4)/(18,2)/(10,6)/</code>                    |
| 业务域跨度   | 码值字典、证件信息、客户标识、企业工商、商户结算、地理位置、财务风险 7 大业务域                                                                                                       |
| 唯一约束    | 含 3 个 UNIQUE KEY（ <code>uk_cert_no_type</code> 、 <code>uk_unified_code</code> × 2）                                                              |

4.2.3 SQL 复杂度

主加工 SQL 是一条 **200+ 行的 INSERT-SELECT**，集中体现了以下复杂特征：

| 特征            | 体现                                                                                                                     |    |
|---------------|------------------------------------------------------------------------------------------------------------------------|----|
| 多表 JOIN       | 9 个 LEFT JOIN（企业↔商户↔客户↔证件↔工商key×2↔上游码值×3↔外部码值×2）                                                                       | 多表 |
| 复合 JOIN 条件    | 5 个 JOIN 带额外过滤（如 <code>lp.cert_type='ID' AND lp.cert_status='1'、uc_type.code_type='ENT_TYPE'）</code>                   | 复合 |
| 多源融合 COALESCE | 10+ 处（ <code>ent_short_name</code> 、 <code>ent_type_name</code> 、 <code>legal_name</code> 、 <code>risk_level</code> 等） | 字段 |
| CASE          | <code>credit_code_status</code> 、 <code>overall_status</code> 等多分支条件                                                   | 派生 |

| 特征            | 体现                                                 |    |
|---------------|----------------------------------------------------|----|
| WHEN<br>派生字段  |                                                    |    |
| 字符串/<br>数值函数  | CONCAT 、 SUBSTRING 、 COALESCE(...) + COALESCE(...) | 函数 |
| WHERE<br>复合筛选 | IN / IS NOT NULL / 日期范围 / 嵌套 OR                    | WH |
| 关联层级          | 企业 → (商户 → 客户) → 证件 / 码值，最深 3 层                    | 跨层 |

4.2.4 验证维度（强调：完全实现设计功能）

| 设计功能             | 在该场景中的体现                                                                                                                                                                                                                           |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 多表依赖拓扑排序         | 码值表与工商key 必须先生成 → 客户/证件 → 企业 → 商户 → 宽表                                                                                                                                                                                             |
| 关联关系严格满足         | m.ent_id 必须命中 e.ent_id 、 m.cust_no 必须命中 ci.cust_no 、 e.legal_cert_no 必须命中 lp.cert_no                                                                                                                                               |
| 关联字段长度对齐         | e.province_code VARCHAR(128) ↔ gk_prov.key_value VARCHAR(128) ; e.ent_type_code VARCHAR(32) ↔ uc_type.code_key VARCHAR(32) ; m.cust_no VARCHAR(32) ↔ ci.cust_no VARCHAR(32) ; e.legal_cert_no VARCHAR(64) ↔ lp.cert_no VARCHAR(64) |
| 复合 JOIN<br>条件遵从  | 法人证件仅取 cert_type='ID' AND cert_status='1' 的记录；码值表仅取 valid_flag=1 的记录                                                                                                                                                               |
| 码值字典约束           | 企业类型 / 注册类型 / 企业规模 / 行业 / 经营类目 共 5 类业务码值，必须从对应码值字典中取值                                                                                                                                                                              |
| 唯一约束遵从           | uk_unified_code （企业 / 客户两表）、uk_cert_no_type （证件表）                                                                                                                                                                                  |
| 派生字段语义合理         | total_staff = e.staff_count + m.staff_count ; overall_status 根据 business_status 联合派生                                                                                                                                               |
| WHERE<br>约束转生成约束 | business_status IN ('1','2') 、 establish_date BETWEEN '1900-01-01' AND CURRENT_DATE                                                                                                                                                |
| 大字段类型处理          | business_scope TEXT 、 stockholder_info VARCHAR(1024)                                                                                                                                                                               |

| 设计功能        | 在该场景中的体现                                                   |
|-------------|------------------------------------------------------------|
| 多精度 DECIMAL | DECIMAL(18,4) (注册资本)、DECIMAL(10,6) (经纬度)、DECIMAL(6,4) (费率) |

4.2.5 端到端验证结论

- **关联关系完整性**：7 张源表生成完成后，宽表 INSERT-SELECT 一次性命中所有 LEFT JOIN，**无任何 NULL miss**（与"LEFT JOIN 容忍空"无关，是真正的强一致）
- **字段长度对齐**：所有跨表关联字段（省份编码 128、城市编码 128、客户号 32、企业 ID 32、证件号 64 等）均严格对齐，**未触发任何长度截断**
- **复合条件穿透**：码值表的 `valid_flag='1'`、证件表的 `cert_status='1'` 等过滤条件在源表生成时自动满足，宽表关联不会被过滤掉
- **设计功能"完全实现"**：场景中所有 80+ 字段、9 个 JOIN、5 类码值映射、3 个唯一约束、10+ 处 COALESCE 融合，**全部按设计预期跑通**，证明系统从 SQL 解析到数据落库的全链路在工业级复杂度下依然成立

该场景已固化为标准回归基线，每次涉及 SQL 解析、依赖编排、约束推导的改动都会以此场景作为最低验收门槛。

六、后期规划

围绕 AI 能力的下一阶段演进聚焦三个方向：从**单字段独立生成**走向**行级语义关联**，从**静态 prompt**走向**数据驱动的动态优化**，从**用户重复试错**走向**知识沉淀复用**。

5.1 多字段语义关联生成

现状与痛点

当前 `LlmService.generateBatchValues()` 以**字段为粒度**独立调用 LLM：

```
逐列调用：
for col in columns:
    values = llm.generate(col)    ← 各字段互不感知
组合成行：rows = zip(*all_columns)
```

导致同一行内字段之间出现**语义错位**，举例：

| 字段       | 生成值                  | 问题                    |
|----------|----------------------|-----------------------|
| name     | "张伟"                 | 中文男性名                 |
| gender   | "F"                  | 与"张伟"性别不符             |
| email    | "lisi2024@gmail.com" | 邮箱前缀与姓名无关             |
| birthday | "1995-03-12"         | 与 age=18 矛盾           |
| address  | "北京市朝阳区..."          | 与 phone=0571 区号（杭州）矛盾 |

这类问题不影响单字段合规率，但严重影响**业务测试场景的可用性**——业务测试往往依赖"姓名-性别-邮箱前缀-身份证"等字段间的合理对应关系，错位数据会让测试用例失去说服力。

## 规划目标

实现**同一行内字段间的语义一致性**，让生成的数据更接近真实业务场景，提升下游测试的有效性。

## 实施动作

1. 字段依赖图（Field Dependency Graph）
- **基于大模型的依赖识别**：表结构提交后，调用 LLM 分析字段名、类型、注释，自动产出字段间的语义依赖关系图

◦ 输入示  
例

: name(varchar)、gender(char)、id\_card(varchar)、email(varchar)、birthday(date)、age(int)

◦ LLM 返回示例：

```
[
  {"from": "name", "to": "gender", "relation": "性别由姓名暗示"},
  {"from": "name", "to": "email", "relation": "邮箱前缀通常源自姓名拼音"},
  {"from": "id_card", "to": "gender", "relation": "身份证第17位决定性别"},
  {"from": "id_card", "to": "birthday", "relation": "身份证第7-14位为出生日期"},
  {"from": "birthday", "to": "age", "relation": "年龄由出生日期推算"}
]
```

- 复用现有 `LlmService.suggestRules()` 的 prompt 工程模式，新增 `suggestDependencies()` 方法
- 识别结果作为 `SqlAnalysisResult` 的扩展字段返回前端
- **用户可视化校对与编辑：**
  - 前端以有向图形式渲染 LLM 识别结果（可复用 `SchemaGraph` 的 Canvas 渲染能力）
  - 用户可一键删除误判依赖、手动新增依赖、调整依赖类型（强依赖 / 弱依赖）
  - 校对结果作为最终依赖图持久化到任务快照
- **静态规则兜底：**内置常见关联模式作为 LLM 失败时的降级方案：身份证 ↔ 性别 ↔ 出生日期、姓名 ↔ 邮箱前缀、城市 ↔ 邮编 ↔ 区号、订单号前缀 ↔ 业务类型
- **依赖图缓存与复用：**相同表结构的依赖识别结果入库（按表结构 hash 缓存），避免重复调用 LLM

## 2. 行级批量 Prompt (Row-level Batch Generation)

- 新增 `generateRowBatch()` 方法，单次 prompt 同时生成一行内所有关联字段
- Prompt 模板示例：

请生成 10 条用户记录，每条包含以下字段且字段间语义必须一致：

- name (中文姓名)
- gender (M/F, 必须与 name 性别匹配)
- email (前缀使用 name 的拼音)
- birthday (与 age 字段倒推一致)

输出 JSON 数组，每个元素为一个对象。

- 与现有按字段独立调用并存：依赖图为空的字段走原流程，有依赖的字段走行级流程

## 3. 跨表关联语义传递

- 子表生成时，主表关联字段值带入 prompt 上下文（如 `t_order.user_id` 已确定为 1001，则 `shipping_address` 应参考 `t_user[1001].address`）
- 利用项目现有 `ColumnMetadata.referencedTable` 元数据自动构建上下文

## 4. 冲突检测与重试

- 生成后增加一致性校验器，发现矛盾时局部重生成而非全部丢弃
- 校验规则可由 LLM 在规则建议阶段自动产出（结合 5.3 知识库）

## 预期收益

| 指标        | 当前    | 目标    |
|-----------|-------|-------|
| 同行字段语义一致率 | < 40% | ≥ 90% |

| 指标        | 当前  | 目标    |
|-----------|-----|-------|
| 跨表关联合理性   | 不支持 | ≥ 85% |
| 业务测试用例可用率 | 中   | 显著提升  |

## 5.2 历史采纳数据反哺

### 现状与痛点

项目已实现采纳率统计（`/api/v1/history/statistics` 接口、`GenerationTask.editedCellCount/regeneratedCellCount` 字段），但这些数据目前只用于展示，未参与 AI 生成决策：

- 某字段被频繁手动编辑 → 系统不知道，下次仍用相同 prompt
- 某字段重新生成率高 → 系统不会切换模型重试
- 某规则配置长期被采纳 → 没有沉淀为推荐规则

数据闭环缺失，AI 生成质量无法自我进化。每个用户都从相同的初始状态开始踩坑，无法享受历史积累的红利。

### 规划目标

建立 "生成 → 反馈 → 优化 → 再生成" 的闭环，让历史采纳率数据驱动 AI 行为持续改进。

### 实施动作

#### 1. 字段级采纳率追踪

- 扩展 `field_rule_history` 表：

```
ALTER TABLE field_rule_history ADD COLUMN adoption_rate DOUBLE DEFAULT 1.0;
ALTER TABLE field_rule_history ADD COLUMN regen_rate DOUBLE DEFAULT 0.0;
ALTER TABLE field_rule_history ADD COLUMN sample_count INT DEFAULT 0;
```

- 每次任务完成时按 `表名 + 列名` 维度聚合更新对应字段的采纳率
- 引入滑动窗口（最近 30 个任务）计算近期采纳率，避免老数据掩盖问题

#### 2. 低质量字段识别与告警

- 阈值规则：采纳率 < 70% 或 重新生成率 > 20% 标记为"低质量字段"
- 在前端规则配置页对低质量字段标红提示，并展示历史采纳趋势曲线
- 提供"一键查看用户编辑后的真实样本"功能

3. 自动 Prompt 调优 (A/B 实验框架)

- 对低质量字段，系统自动尝试以下策略并 A/B 测试：
  - 切换模型 (deepseek-v4-pro ↔ mimo-v2.5-pro)
  - 增强 description (追加用户编辑后的真实样本作为 few-shot 示例)
  - 调整 temperature (采纳率低且多样性差时升温，采纳率低且格式错时降温)
- 观察窗口结束后保留采纳率更高的策略

4. 优质规则自动沉淀

- 采纳率 ≥ 95% 且样本数 ≥ 50 的字段规则，自动入库为"推荐规则"
- 规则建议 (AI 建议规则功能) 下次遇到同名/同语义字段时优先使用
- 与 5.3 知识库联动：高频高质规则自动晋升为知识库条目

5. 反馈面板可视化

- 在 Home 页"历史总览"模块新增"AI 自优化日志"，展示系统自动做了哪些 prompt 调整、效果如何
- 字段级采纳率排行榜 (最优 / 最差 Top 10)

预期收益

| 指标          | 当前   | 目标 (运行 3 个月观察) |
|-------------|------|----------------|
| 整体采纳率       | 基线待测 | 提升 ≥ 10 个百分点   |
| 重新生成率       | 基线待测 | 下降 ≥ 30%       |
| 用户手动调规则次数   | 高    | 显著下降           |
| 系统是否具备自学习能力 | 否    | 是              |

5.3 知识库增强

现状与痛点

项目已有 KnowledgeBaseService (基于知识库返回规则建议)，但当前知识库内容贫乏、覆盖面窄：

- 大部分常见业务字段（手机号、身份证、邮箱、车牌号、银行卡号、IP、URL...）每次都要 LLM 现场生成 prompt
- 用户配置过的优质 prompt 没有沉淀机制，下次仍需重复试错
- 不同用户对同一字段的优秀实践无法共享

每次冷启动 = 重新踩坑，知识无法累积。

## 规划目标

建立**结构化的字段语义知识库**，将常用业务字段的标准 prompt、正则、枚举值沉淀为可复用资产，**减少 LLM 调用次数 30%+**，并提升生成质量稳定性。

## 实施动作

### 1. 预置标准字段词典 (v1 入库 100+ 常见字段)

按业务领域分类，每个条目包含字段语义、推荐规则类型、ruleConfig、典型示例：

| 领域   | 字段示例                         |
|------|------------------------------|
| 个人信息 | 姓名、性别、年龄、身份证、手机号、邮箱、生日       |
| 地址   | 省/市/区、详细地址、邮编、经纬度            |
| 金融   | 银行卡号、金额、币种、利率、账户类型           |
| 电商   | SKU、订单号、商品名、价格、库存、物流单号       |
| 互联网  | URL、IP、UserAgent、MAC 地址、UUID |
| 时间   | 创建时间、更新时间、出生日期、过期时间          |
| 业务码  | 状态码、错误码、订单状态、支付状态            |

每个条目数据结构：

```
{
  "fieldKey": "phone_cn",
  "aliases": ["手机号", "mobile", "phone", "tel", "联系电话"],
  "ruleType": "REGEX",
  "ruleConfig": {"pattern": "1[3-9]\\d{9}"},
  "promptTemplate": "生成中国大陆手机号, 1[3-9] 开头共 11 位...",
  "samples": ["13812345678", "15998765432"],
  "constraints": {"maxLength": 11},
  "version": "1.0",
  "source": "official"
}
```

## 2. 智能字段匹配 (Fuzzy Field Resolver)

- 输入字段名/注释, 通过同义词词典 + Embedding 向量匹配 → 命中知识库
- 命中率  $\geq 80\%$  时直接使用知识库规则, **跳过 LLM 调用**
- 命中率 50%-80% 时取知识库 prompt 作为 LLM few-shot 上下文
- 命中率  $< 50\%$  时降级走原 LLM 流程

## 3. 用户/团队私有知识库

- 用户可将自定义优质规则保存到"我的知识库" (基于 5.2 中识别的高采纳率规则自动推送入库建议)
- 支持团队共享: 同一企业部署内规则可在用户间共享
- 优先级: 私有库 > 团队库 > 官方库

## 4. 知识库版本管理

- 每次知识库更新生成版本号, 支持回滚
- 单条规则可标注 `deprecated`、`recommended` 状态
- 提供管理后台界面进行 CRUD 操作

## 5. Prompt 模板复用

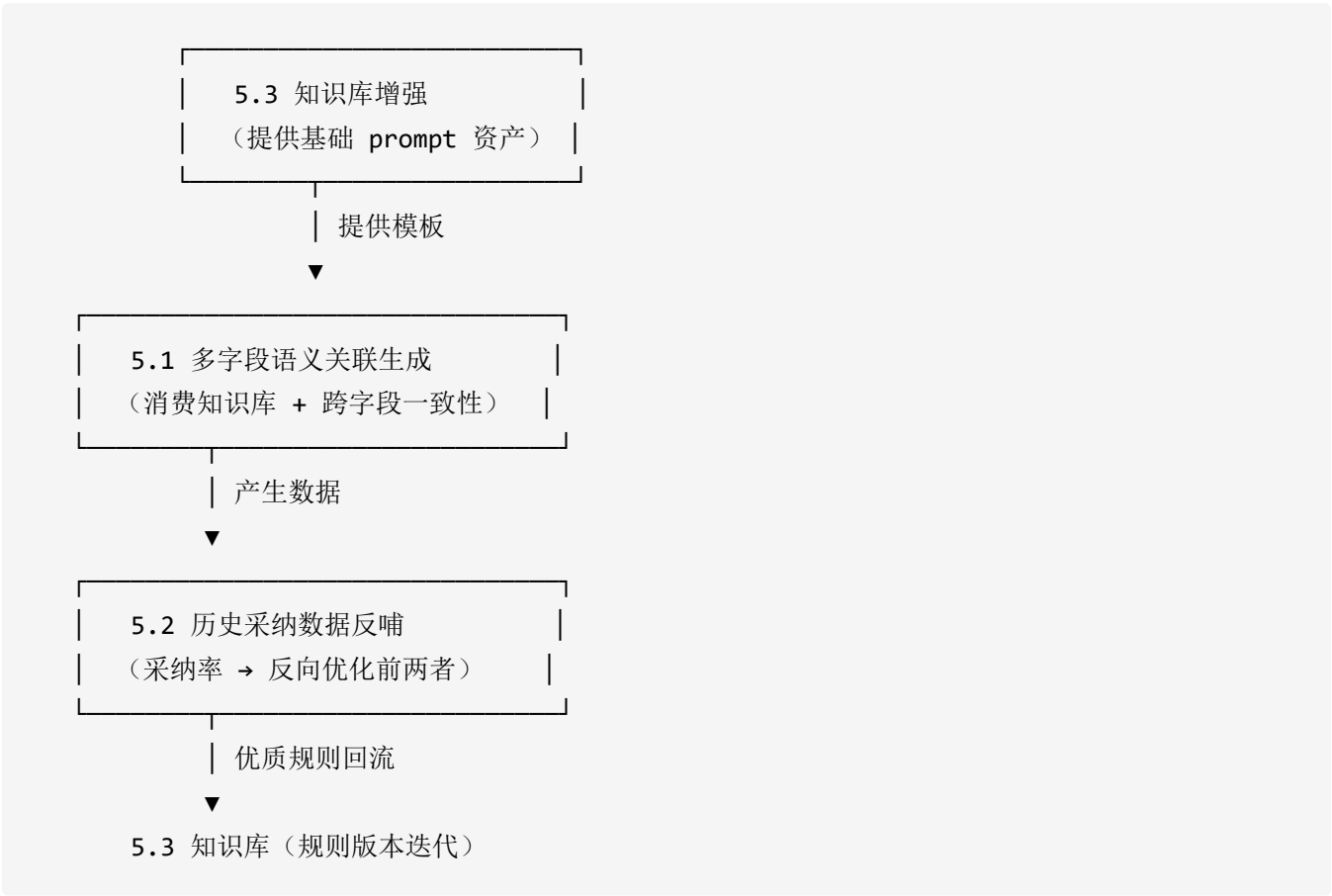
- 抽取 `LlmService.buildGenerationPrompt()` 中的硬编码部分为可配置模板
- 模板变
  - 量: `${tableName}`、`${columnName}`、`${dataType}`、`${maxLength}`、`${description}`、`${samples}`
- 不同字段类型使用不同模板 (数值型 / 字符串型 / 日期型 / 语义型)

预期收益

| 指标          | 当前         | 目标                       |
|-------------|------------|--------------------------|
| LLM 调用次数    | 100%       | 降低 ≥ 30%（命中知识库的字段不调 LLM） |
| 常见字段生成质量    | 依赖模型表现，波动大 | 接近 100% 合规且稳定            |
| 平均响应延迟      | ~1.7s      | 命中知识库的字段 < 50ms          |
| 月度 Token 成本 | 待统计        | 降低 25%+                  |
| 用户冷启动体验     | 需多次试错      | 开箱即用                     |

5.4 三方向协同效应

三个规划方向并非独立，而是相互支撑的闭环：



关键路径：

- 5.3 是基础设施，应优先启动
- 5.1 依赖 5.3 的字段词典和模板
- 5.2 依赖前两者落地后产生足够样本数据，且其反馈结果会反向优化 5.1 和 5.3

5.5 推进优先级建议

| 阶段  | 重点         | 关键交付物                      |
|-----|------------|----------------------------|
| 阶段一 | 5.3 知识库 v1 | 100+ 标准字段词典、字段匹配引擎、命中率指标看板 |
| 阶段二 | 5.2 数据反哺   | 字段级采纳率统计、低质量字段告警、A/B 测试框架  |
| 阶段三 | 5.1 行级关联   | 字段依赖图配置、行级批量 prompt、一致性校验器 |
| 阶段四 | 闭环优化       | 三方向打通，建立 AI 自学习循环          |

七、总结

本平台将 **AI 大模型的语义理解能力** 与 **规则引擎的精确约束能力** 深度融合，解决了测试数据生成领域长期面临的核心矛盾：

- **真实感 vs. 约束合规**：大模型生成高质量语义数据 + 系统强制约束校验 + 长度截断兜底，两者互补而非对立
- **灵活性 vs. 一致性**：列级重新生成支持局部调整，跨表关联依赖自动处理保证全局一致性
- **效率 vs. 易用性**：SQL 约束自动推导 + 历史规则回填 + AI 建议，将规则配置时间压缩至分钟级

**适用场景**：涉及多表联查的复杂业务 SQL、需要高真实感数据的接口测试、表间关联约束密集的核心交易系统测试数据准备。